

MEMORIA TÉCNICA: DETECCIÓN DE FRAUDE EN TRANSACCIONES FINANCIERAS

Guía Metodológica Paso a Paso: Limpieza, Modelado Relacional y Consultas Avanzadas en SQL Server

Motivo de elección :

<https://www.kaggle.com/datasets/computingvictor/transactions-fraud-datasets>

Al enfrentarme a la selección del proyecto final para el Bootcamp, me encontraba en una etapa donde cualquier temática representaba un verdadero desafío debido a mi reciente incursión en el mundo del Análisis de Datos y el Machine Learning. Ante la diversidad de opciones, decidí buscar una orientación estratégica para asegurar que el proyecto no solo cumpliera con los requisitos académicos, sino que también se convirtiera en una pieza de portafolio valiosa y con proyección en el mercado laboral.

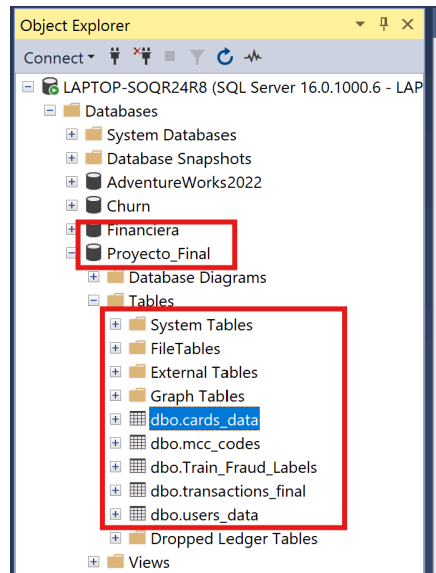
A través de la recomendación de un profesional Senior del sector —mi hermano—, identifiqué que el sector financiero y bancario es una de las áreas con mayor demanda analítica y estabilidad laboral. Con este enfoque en mente, exploré diversas alternativas en Kaggle. Aunque inicialmente evalué temáticas como la predicción de fuga de clientes (*churn*), el análisis de **transacciones fraudulentas** despertó un interés superior, ya que representaba un problema complejo, dinámico y con un impacto directo en la operativa de negocio.

La elección específica de este dataset de transacciones respondió a criterios de capacidad y complejidad técnica. Necesitaba un conjunto de datos masivo, con un elevado número de registros y variables suficientes para aplicar técnicas avanzadas de preparación de datos, *feature engineering* y modelado predictivo.

Fue así como llegué al conjunto de datos Transactions Fraud Datasets de Victor Computo, una opción que la inteligencia artificial me recomendó al evaluar mis necesidades del proyecto. Este dataset resultó ser la elección idónea no por un motivo arbitrario, sino porque reunía la estructura idónea para poner a prueba todo lo aprendido durante el Bootcamp, ofreciendo una base masiva de dos millones y medio de registros distribuidos en tablas interconectadas de usuarios, tarjetas y movimientos. Esta riqueza de información me permitió trabajar con datos socioeconómicos, límites de crédito, canales de pago como chip o banda magnética y registros de errores operativos, brindándome el escenario perfecto para aplicar técnicas avanzadas de preparación de datos, ingeniería de variables y algoritmos de detección de anomalías en un entorno analítico de escala real.

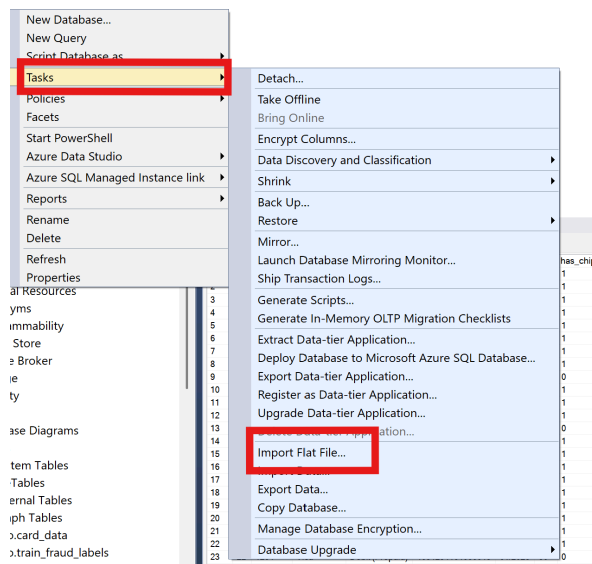
Paso 1: Crear una base de datos llamada "Proyecto_final" y cargar los archivos CSV

Primero, abrimos **Microsoft SQL Server Management Studio (SSMS)** y nos conectamos a la instancia local. Hacemos clic derecho sobre la carpeta *Databases*, seleccionamos *New Database...* y creamos la base de datos con el nombre **Proyecto_final**. Esta base de datos contendrá las tablas transaccionales, tarjetas, usuarios, mcc y fraudes.



Paso 2: Importación de datos a SQL

Segundo, sobre la base de datos **Proyecto_final**, hacemos clic derecho, nos dirigimos a *Tasks* y seleccionamos la opción **Import Flat File...** para subir los archivos *transactions_data.csv*, *cards_data.csv* y *users_data.csv*.



Utilice la herramienta de exploración para ubicar el archivo fuente CSV en el disco duro y proceder con

su carga en la plataforma.

Specify Input File
This operation will create a table from your input file.

Location of file to be imported

New table name:

Table schema:



Durante el proceso de importación mediante el asistente de SSMS, la interfaz despliega la sección **Preview Data**, la cual permite visualizar de forma tabular las primeras cincuenta filas del archivo fuente CSV. Esta vista previa sirve como mecanismo de control para corroborar la correcta delimitación de columnas clave (tales como **id**, **client_id**, **card_brand**, **card_type**, entre otras) y comprobar que la asignación automática de tipos de datos es coherente con el contenido tabular previo a su persistencia en el servidor.

Introduction
Specify Input File
Preview Data
Modify Columns
Summary
Results

Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

id	client_id	card_brand	card_type	card_number	expires	cvv	has_chip	num_cards	credit_limit	acct...
4524	825	Visa	Debit	4344...	12/20...	623	YES	2	\$24295	09/20
2731	825	Visa	Debit	4956...	12/20...	393	YES	2	\$21968	04/20
3701	825	Visa	Debit	4582...	02/20...	719	YES	2	\$46414	07/20
42	825	Visa	Credit	4879...	08/20...	693	NO	1	\$12400	01/20
4659	825	Mast...	Debit...	5722...	03/20...	75	YES	1	\$28	09/20
4537	1746	Visa	Credit	4404...	09/20...	736	YES	1	\$27500	09/20
1278	1746	Visa	Debit	4001...	07/20...	972	YES	2	\$28508	02/20
3687	1746	Mast...	Debit	5627...	06/20...	48	YES	2	\$9022	07/20
3465	1746	Mast...	Debit...	5711...	11/20...	722	YES	2	\$54	06/20
3754	1746	Mast...	Debit...	5766...	02/20...	908	YES	1	\$99	07/20
5144	1718	Mast...	Debit	5495...	03/20...	677	YES	2	\$31599	10/20
2029	1718	Mast...	Debit	5804...	07/20...	258	NO	2	\$27480	03/20
2379	1718	Mast...	Debit	5766...	02/20...	992	YES	1	\$26743	03/20
2732	1718	Visa	Debit	4242...	06/20...	928	YES	1	\$31463	04/20
4706	1718	Mast...	Debit	5191...	06/20...	360	YES	1	\$16055	09/20
281	708	Visa	Credit	4017...	05/20...	877	YES	2	\$98100	01/20
1106	708	Mast...	Debit...	5581...	06/20...	448	YES	1	\$62	02/20
5621	708	Visa	Debit	4032...	06/20...	53	YES	1	\$132...	11/20
5165	708	Visa	Debit	4935...	06/20...	649	YES	1	\$125...	10/20
5757	1164	Mast...	Debit	5905...	01/20...	886	YES	1	\$34900	12/19
27	68	Visa	Credit	4099...	09/20...	74	YES	2	\$9900	01/20
4648	68	Visa	Debit	4281...	07/20...	865	YES	2	\$21587	09/20
5435	68	Visa	Credit	4411...	11/20...	922	YES	2	\$15800	11/20
3287	1075	Mast...	Debit	5202...	04/20...	861	YES	1	\$29964	06/20
4761	1075	Mast...	Credit	5080...	07/20...	686	YES	2	\$11200	08/20

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

En la sección **Modify Columns** del asistente de importación, se auditan y ajustan los tipos de datos asignados a cada campo (como **smallint**, **nvarchar**, **bigint**, **money**, entre otros) para asegurar la

compatibilidad con el motor de base de datos. Desde esta misma interfaz se seleccionan explícitamente los campos identificadores (como `client_id`) para habilitar la casilla **Primary Key**, garantizando de este modo la unicidad de los registros y la integridad referencial en la tabla resultante.

Modify Columns

This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	Allow Nulls
id	smallint	☒	☐
client_id	smallint	☒	☐
card_brand	nvarchar(50)	☐	☐
card_type	nvarchar(50)	☐	☐
card_number	bigint	☐	☐
expires	nvarchar(50)	☐	☐
cvv	smallint	☐	☐
has_chip	bit	☐	☐
num_cards_issued	tinyint	☐	☐
credit_limit	money	☐	☐
acct_open_date	nvarchar(50)	☐	☐
year_pin_last_changed	smallint	☐	☐
card_on_dark_web	nvarchar(50)	☐	☐

La sección **Summary** presenta un desglose detallado con los parámetros configurados antes de la ejecución definitiva, mostrando el servidor de destino, la base de datos seleccionada, el nombre de la nueva tabla y la ruta de origen del archivo. Al pulsar el botón *Finish* desde esta ventana, se procesa la carga masiva de los registros garantizando que toda la estructura definida previamente se aplique de forma correcta en el servidor.

Summary

To complete the operation using the specified inputs, click Finish.

Import Information

- Name: LAPTOP-SOQR24R8
- Database Name: prueba
- Table Name: dbo.card_data
- File to be imported: C:\Users\yutto\OneDrive\Escritorio\cards_data.csv

La pantalla final de **Operation Complete** muestra el estado del proceso de carga masiva, confirmando mediante el indicador de éxito (**Success**) que el volcado de los registros se ha completado de forma satisfactoria en la tabla de destino del servidor.

✓ Operation Complete

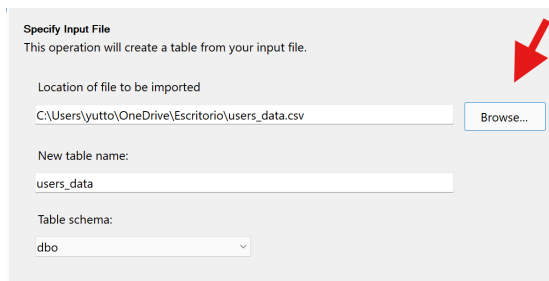
Summary:

Name	Result
✓ Insert Data	Success

Así repetimos el mismo proceso con las otras dos tablas restantes.

Paso 3: Definición de las columnas que contiene el dataset

Una vez completada la ingesta de los archivos fuente en el servidor de base de datos, se procede a detallar la arquitectura del modelo relacional. El sistema se compone de diversas entidades normalizadas que almacenan el comportamiento transaccional, el perfil de los usuarios, las características de los instrumentos financieros y los catálogos auxiliares de apoyo. A continuación, se describe la estructura y el propósito analítico de cada tabla:



Specify Input File
This operation will create a table from your input file.

Location of file to be imported
C:\Users\yutto\OneDrive\Escritorio\users_data.csv [Browse...](#)

New table name:
users_data

Table schema:
dbo

Esta tabla recopila el perfil sociodemográfico e historial crediticio básico de cada cliente registrado en el sistema, permitiendo cruzar variables personales con la actividad transaccional y los incidentes de fraude.

3.1 El esquema de la tabla almacena atributos clave que caracterizan a cada usuario:

	id	current_age	retirement_age	birth_year	birth_month	gender	address	latitude	longitude	per_capita_income	yearly_income	total_debt	credit_score	num_credit_cards
1	1	43	74	1976	4	Female	113 Burns Lane	3044	-8718	22247,00	45360,00	14587,00	704	3
2	2	48	64	1971	8	Male	6035 Forest Avenue	4084	-7387	13461,00	27447,00	80850,00	673	5
3	3	49	65	1970	12	Male	840 Elm Avenue	3389	-9851	13705,00	27943,00	18693,00	681	4
4	4	54	72	1965	3	Female	6016 Little Creek Boulevard	4761	-1223	37485,00	76431,00	115362,00	716	5

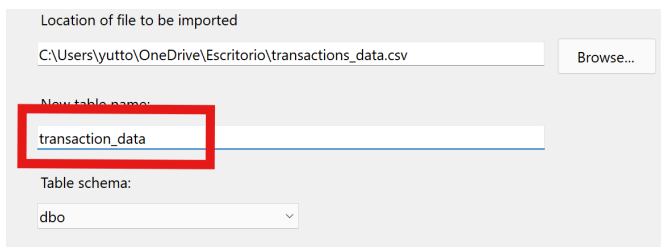
- **Id (Identificador del Cliente):** Clave primaria numérica que vincula de forma unívoca al titular con sus respectivas tarjetas registradas en el sistema.
- **Current_age (Edad Actual):** Edad del usuario al momento del análisis, permitiendo evaluar correlaciones entre rangos etarios y vulnerabilidad ante operaciones ilícitas.
- **Retirement_age (Edad de Jubilación):** Proyección o estatus laboral del cliente respecto a su

etapa de retiro financiero.

- **Birth_year y birth_month (Año y Mes de Nacimiento):** Variables temporales de control para validar la consistencia de la edad y la demografía de la base de datos.
- **Gender (Género):** Variable categórica que identifica el género del titular (**Male, Female**), empleada en análisis de segmentación demográfica del fraude.
- **Address, city, state, zip (Ubicación Geográfica):** Datos de residencia del usuario (dirección, ciudad, estado y código postal), útiles para contrastar la residencia habitual frente al lugar geográfico donde se ejecutan las transacciones.
- **Per_capita_income (Ingreso Per Cápita):** Estimación económica del poder adquisitivo o nivel de ingresos de la zona geográfica del cliente.
- **Yearly_income (Ingreso Anual):** Capacidad financiera declarada o estimada del titular.
- **Total_debt (Deuda Total):** Nivel de endeudamiento histórico acumulado del usuario.
- **Credit_score (Puntuación de Crédito):** Calificación crediticia central que mide la solvencia y el riesgo financiero asociado al perfil del cliente.
- **Num_credit_cards (Número de Tarjetas de Crédito):** Cantidad de instrumentos financieros activos que posee el usuario bajo su titularidad.

La inclusión de **users_data** en el núcleo relacional dota al proyecto de una perspectiva sociodemográfica y de riesgo crediticio indispensable. Permite analizar si factores como el nivel de ingresos (**yearly_income**), la puntuación de crédito (**credit_score**), la edad o la ubicación geográfica influyen de manera directa en la propensión a sufrir fraudes, transformando simples registros transaccionales en un modelo de comportamiento predictivo y de auditoría integral.

En la siguiente tabla:



The screenshot shows a data import configuration window. It has three main sections: 'Location of file to be imported' with a text box containing 'C:\Users\yutto\OneDrive\Escritorio\transactions_data.csv' and a 'Browse...' button; 'New table name:' with a text box containing 'transaction_data' which is highlighted by a red rectangle; and 'Table schema:' with a dropdown menu showing 'dbo'.

El núcleo operativo del modelo se sustenta en la tabla de transacciones, la cual registra de forma cronológica todas las operaciones financieras realizadas con las tarjetas de crédito y débito. Esta entidad masiva concentra la información detallada de cada movimiento, permitiendo auditar tanto importes como ubicaciones, métodos de autenticación y códigos comerciales.

3.2 El esquema normalizado de la tabla procesada comprende los siguientes atributos clave:

	id	date	client_id	card_id	amount	use_chip	merchant_id	merchant_city	merchant_state	zip	mcc	errors
1	7719730	2010-03-03 19:33:00	1772	5918	\$50.73	Swipe Transaction	55060	Rancho Cucamonga	CA	91730.0	5812	
2	7719731	2010-03-03 19:34:00	1138	2122	\$46.99	Swipe Transaction	85434	Odenville	AL	35120.0	5310	
3	7719732	2010-03-03 19:35:00	1077	2539	\$28.19	Swipe Transaction	46284	Ukiah	CA	95482.0	5411	
4	7719733	2010-03-03 19:36:00	1098	2253	\$95.00	Swipe Transaction	59935	Granville	IA	51022.0	5499	
5	7719734	2010-03-03 19:36:00	1915	2113	\$110.62	Swipe Transaction	19964	Monsey	NY	10952.0	5311	
6	7719735	2010-03-03 19:37:00	487	2825	\$51.45	Swipe Transaction	73160	Albany	NY	12202.0	5411	

- **Transaction_id (Identificador de la Transacción):** Clave primaria numérica que otorga un carácter unívoco e irrepitable a cada operación registrada en el sistema.
- **Fecha y hora (Variables Temporales):** Desglose temporal obtenido de la estandarización del campo original, el cual permite analizar la secuencialidad, patrones horarios y rachas operativas previas a un ilícito.
- **Client_id (Identificador del Cliente):** Clave foránea que vincula la operación directamente con el titular propietario del instrumento financiero.
- **Card_id (Identificador de la Tarjeta):** Clave foránea que especifica qué plástico o cuenta de tarjeta concreta fue utilizada para ejecutar el movimiento económico.
- **Amount (Importe de la Transacción):** Valor monetario de la operación, empleado como variable crítica para la detección de anomalías y percentiles de gasto atípico.
- **Use_chip (Método de Uso del Chip):** Variable categórica que indica si la transacción se realizó mediante inserción de chip, banda magnética o de forma presencial/online, aportando contexto sobre el nivel de seguridad aplicado en el punto de venta.
- **Merchant_id (Identificador del Comercio):** Código que identifica de manera unívoca al establecimiento comercial donde se efectuó el cobro.
- **Merchant_city y Merchant_state (Ubicación del Comercio):** Ciudad y estado geográfico del establecimiento, utilizados para contrastar la distancia entre el lugar de residencia del usuario y el punto físico de consumo.
- **Zip (Código Postal del Comercio):** Localización postal normalizada del negocio.
- **Mcc (Código de Categoría Comercial):** Enlace directo con el catálogo de comercios ([mcc_codes](#)) para determinar el sector económico de la actividad (ej. grandes almacenes, gasolineras, transferencias).
- **Error_type (Tipo de Error):** Registro de incidencias técnicas o rechazos previos en la pasarela de pagos (en caso de presentarse anomalías sistémicas).

La tabla de transacciones constituye la columna vertebral analítica del proyecto. Su estructura detallada permite ejecutar algoritmos complejos y consultas avanzadas. Gracias a esto, es posible correlacionar el comportamiento transaccional diario con las etiquetas de fraude, facilitando la identificación precisa de patrones de riesgo y anomalías financieras.

Y finalmente la última tabla en formato csv:

Specify Input File
This operation will create a table from your input file.

Location of file to be imported
C:\Users\yutto\OneDrive\Escritorio\cards_data.csv

New table name:

Table schema:
dbo

La entidad **cards_data** recopila el inventario detallado de todos los instrumentos de pago (tarjetas de crédito y débito) emitidos a nombre de los clientes. Esta tabla es fundamental para vincular el comportamiento transaccional con las características técnicas, los límites de gasto y los niveles de seguridad de cada plástico operativo.

3.3 El esquema de la tabla almacena atributos clave que caracterizan a cada tarjeta registrada:

	id	client_id	card_brand	card_type	card_number	expires	cvv	has_chip	num_cards_issued	credit_limit	acct_open_date	year_pin_last_changed	card_on_dark_web
2	1	550	Mastercard	Credit	5278231764792292	06/2024	396	1	1	11600,00	01/1994	2013	No
3	2	556	Mastercard	Debit	5889825928297675	09/2021	422	1	1	19948,00	01/1995	2011	No
4	3	1937	Visa	Credit	4289888672554714	04/2020	736	1	2	16400,00	01/1995	2015	No
5	4	1001	Mastercard	Debit	5432366078599045	03/2024	590	1	2	10420,00	01/1997	2007	No

- **Id (Identificador de la Tarjeta):** Clave primaria numérica que otorga un código unívoco a cada plástico, sirviendo como vínculo relacional directo con la tabla de transacciones (**card_id**).
- **Client_id (Identificador del Cliente):** Clave foránea que asocia la tarjeta con su respectivo titular, permitiendo establecer una relación uno a muchos (1:N) donde un cliente puede poseer múltiples instrumentos financieros.
- **Card_brand (Marca de la Tarjeta):** Entidad o red de procesamiento emisora del plástico (por ejemplo, Visa, Mastercard, etc.), útil para evaluar la distribución de operaciones y posibles vulnerabilidades por marca.
- **Card_type (Tipo de Tarjeta):** Variable categórica que especifica si el instrumento opera bajo la modalidad de crédito (**Credit**) o débito (**Debit**), un factor analítico clave evidenciado en los cruces matriciales de fraude.
- **Card_number (Número de Tarjeta):** Registro de identificación del plástico (enmascarado o estructurado) asociado a la cuenta.
- **Expires (Fecha de Expiración):** Período de vigencia del instrumento de pago (mes y año).
- **Cvv (Código de Seguridad CVV):** Valor de verificación de la tarjeta, empleado en los protocolos de validación para transacciones no presenciales o en línea.
- **Has_chip (Disponibilidad de Chip):** Indicador binario (verdadero/falso) que señala si la tarjeta cuenta con tecnología de circuito integrado EMV para mayor seguridad en el punto de venta.
- **Num_cards_issued (Número de Tarjetas Emitidas):** Cantidad total de plásticos expedidos

históricamente asociados a esa cuenta o titularidad.

- **Credit_limit (Límite de Crédito):** Monto máximo de endeudamiento autorizado para la tarjeta, empleado para evaluar el riesgo de sobregiro o exposición ante fraudes de alta cuantía.
- **Acct_open_date (Fecha de Apertura de Cuenta):** Indicador temporal que refleja la antigüedad del instrumento financiero en el sistema.
- **Year_pin_last_changed (Año de Última Modificación del PIN):** Variable de seguridad que registra cuándo se actualizó por última vez la clave de acceso del instrumento.
- **Card_on_dark_web (Exposición en la Dark Web):** Indicador de alerta temprana que señala si el número o los datos de la tarjeta han sido detectados en filtraciones de información o mercados negros digitales.

La integración de **cards_data** permite enriquecer sustancialmente el análisis de riesgo al aportar variables críticas de infraestructura y seguridad. Gracias a esto, es posible determinar qué tipos de tarjetas presentan mayores tasas de incidencia delictiva y evaluar cómo factores como la modalidad del plástico o sus protocolos de protección influyen directamente en la actividad fraudulenta.

Paso 4: Ingesta del catálogo comercial "mcc_codes.json" mediante T-SQL

Para evitar conflictos de tipo LOB con archivos JSON, primero creamos la tabla **mcc_codes** asignando la columna **cm** como **llave primaria (PRIMARY KEY)** para evitar duplicados. Segundo, procesamos el archivo JSON estructurado mediante las funciones nativas **OPENROWSET** y **OPENJSON**:

Para ello, se utilizarán las siguientes instrucciones:

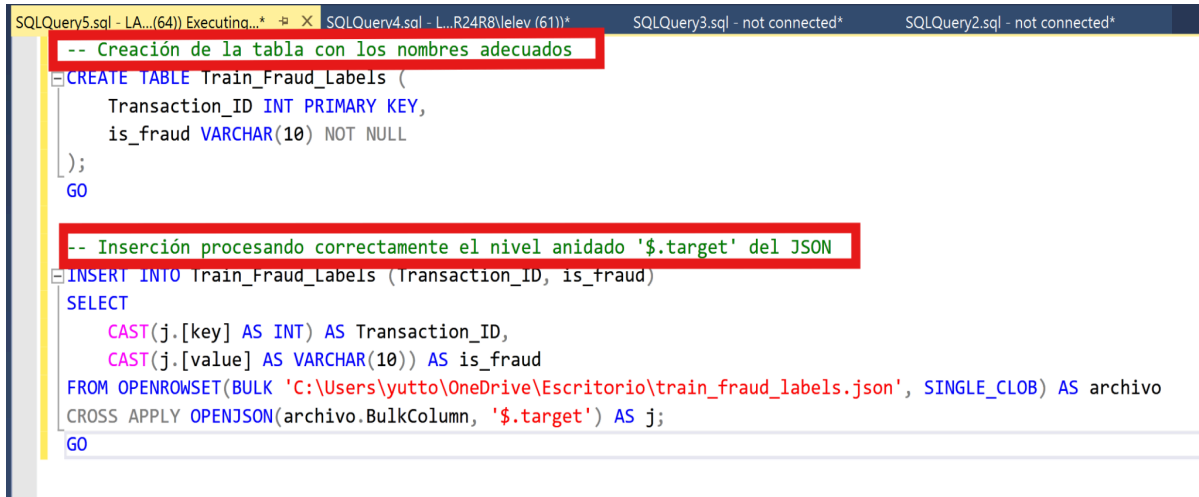
Para integrar la variable objetivo (etiquetas de fraude) contenida en el archivo **train_fraud_labels.json**, el cual presenta una estructura anidada bajo la clave raíz **"target"**, se diseñó un procedimiento automatizado en T-SQL. Este script garantiza la correcta conversión de los pares clave-valor del formato NoSQL a un modelo relacional normalizado en Microsoft SQL Server.

```
train_fraud_labels.json X
C: > Users > yutto > OneDrive > Escritorio > train_fraud_labels.json
1 [{"target": {"10649266": "No", "23410063": "No", "9316588": "No", "12478022": "No", "9558530": "No", "12532830": "No", "19526;
```

El procedimiento implementado contempla la validación de existencia de la entidad, la creación de la tabla relacional con su respectiva clave primaria y el procesamiento de los nodos anidados mediante las funciones nativas **OPENROWSET** y **OPENJSON**.

La ejecución exitosa de esta rutina permite consolidar la **variable objetivo (etiqueta de clase)** dentro del servidor de bases de datos. Al transformar los identificadores y sus estados asociados (**Yes** o **No**) en la entidad relacional **Train_Fraud_Labels**, se completa el núcleo del modelo analítico. Esto habilita la ejecución de cruces avanzados con las tablas transaccionales, de tarjetas y comercios, permitiendo

realizar auditorías de comportamiento, análisis de percentiles de riesgo y la evaluación de patrones en algoritmos de detección de fraude.



```
-- Creación de la tabla con los nombres adecuados
CREATE TABLE Train_Fraud_Labels (
    Transaction_ID INT PRIMARY KEY,
    is_fraud VARCHAR(10) NOT NULL
);
GO

-- Inserción procesando correctamente el nivel anidado '$.target' del JSON
INSERT INTO Train_Fraud_Labels (Transaction_ID, is_fraud)
SELECT
    CAST(j.[key] AS INT) AS Transaction_ID,
    CAST(j.[value] AS VARCHAR(10)) AS is_fraud
FROM OPENROWSET(BULK 'C:\Users\yutto\OneDrive\Escritorio\train_fraud_labels.json', SINGLE_CLOB) AS archivo
CROSS APPLY OPENJSON(archivo.BulkColumn, '$.target') AS j;
GO
```

Aquí se puede comprobar que el código se ha ejecutado correctamente:

(8914963 rows affected)

Completion time: 2026-09-04T20:46:37.8932589+02:00

El esquema de la tabla de fraude almacena los atributos clave que caracterizan y etiquetan cada evento de riesgo registrado.

- **Transaction_ID (Identificador de la Transacción):** Clave primaria o identificador numérico unívoco que actúa como vínculo relacional directo con la tabla principal de transacciones, permitiendo asociar cada operación con su respectivo veredicto de seguridad.
- **Is_fraud (Etiqueta de Fraude):** Variable objetivo (*target*) de naturaleza categórica o binaria (con valores como 'Yes' o 'No') que determina de forma explícita si el evento analizado corresponde o no a una transacción fraudulenta, sirviendo como base fundamental para el entrenamiento de los modelos de Machine Learning supervisado.

La integración de la tabla de etiquetas de fraude resulta fundamental para habilitar el enfoque de aprendizaje supervisado y la analítica predictiva, al aportar la variable objetivo indispensable (*is_fraud*) que clasifica de manera fehaciente los eventos ilícitos. Gracias a esto, es posible entrenar modelos de Machine Learning con alta precisión, aislar patrones de comportamiento anómalo dentro de las transacciones y medir con exactitud el impacto del riesgo delictivo en toda la infraestructura

financiera del proyecto.

	Transaction_ID	Is_Fraud
1	7475327	No
2	7475328	No
3	7475329	No

De forma complementaria al procesamiento de las etiquetas de riesgo, se integró el archivo `mcc_codes.json`, el cual almacena la taxonomía de los códigos de categoría comercial (MCC) y sus respectivas descripciones en una estructura de pares clave-valor.

Para evitar conflictos de restricciones en columnas de gran tamaño y unificar la tipología de ingesta basada en documentos semiestructurados, se implementó al igual que la tabla anterior, el siguiente procedimiento mediante consultas T-SQL utilizando las funciones nativas `OPENROWSET` y `OPENJSON`:

```
CREATE TABLE mcc_codes (  
    cmm VARCHAR(10) PRIMARY KEY,  
    descripcion VARCHAR(255) NOT NULL  
);  
GO  
  
INSERT INTO mcc_codes (cmm, descripcion)  
SELECT  
    CAST(j.[key] AS VARCHAR(10)) AS cmm,  
    CAST(j.[value] AS VARCHAR(255)) AS descripcion  
FROM OPENROWSET(BULK 'C:\Data\mcc_codes.json', SINGLE_CLOB) AS archivo  
CROSS APPLY OPENJSON(archivo.BulkColumn) AS j;  
GO
```

La ejecución de esta rutina permite persistir el catálogo descriptivo de establecimientos comerciales dentro del servidor. Esta entidad actúa como un puente categórico fundamental para las consultas analíticas avanzadas, permitiendo traducir los códigos numéricos de los comercios en sectores de actividad legibles (por ejemplo, identificar transacciones en grandes almacenes o gasolineras) y evaluar con mayor precisión los rubros económicos más vulnerables frente al fraude.

	cmm	descripcion
1	1711	Heating, Plumbing, Air Conditioning Contractors
2	3000	Steelworks
3	3001	Steel Products Manufacturing
4	3005	Miscellaneous Metal Fabrication
5	3006	Miscellaneous Fabricated Metal Products
6	3007	Coated and Laminated Products
7	3008	Steel Drums and Barrels
8	3009	Fabricated Structural Metal Products
9	3058	Tools, Parts, Supplies Manufacturing
10	3066	Miscellaneous Metals
11	3075	Bolt, Nut, Screw, Rivet Manufacturing

Paso 3: Limpieza y estandarización en la tabla "transactions_final"

En la tabla de transacciones original "transactions_data" se identificaron campos que requerían limpieza antes del análisis.

	id	date	client_id	card_id	amount	use_chip	merchant_id	merchant_city	merchant_state	zip	mcc	errors
1	7719730	2010-03-03 19:34:00	1172	2122	\$66.75	Swipe Transaction	85434	Nashville	GA	37203.0	5812	
2	7719731	2010-03-03 19:34:00	1138	2122	\$46.99	Swipe Transaction	85434	Odenville	AL	35120.0	5310	
3	7719732	2010-03-03 19:35:00	1077	2539	\$28.19	Swipe Transaction	46284	Ukiah	CA	95482.0	5411	
4	7719733	2010-03-03 19:36:00	1098	2253	\$95.00	Swipe Transaction	59935	Granville	IA	51022.0	5499	

1.1 Descomposición de la columna Date en Hora y fecha

```
-- 1. Añadir las dos nuevas columnas a la tabla existente
ALTER TABLE transactions_data
ADD fecha DATE,
    hora TIME;
GO

-- 2. Rellenar las nuevas columnas extrayendo la información del campo 'date' original
UPDATE transactions_data
SET fecha = CAST([date] AS DATE),
    hora = CAST([date] AS TIME);
GO
```

Tras la ejecución del script, el gestor confirma la actualización de los registros:

(2551145 rows affected)

Completion time: 2026-09-05T11:55:27.3603905+02:00

Se comprueba que los valores de fecha y hora han sido extraídos correctamente en sus respectivas columnas:

date	fecha	hora
2010-03-03 19:33:00	2010-03-03	19:33.00000000
2010-03-03 19:34:00	2010-03-03	19:34.00000000
2010-03-03 19:35:00	2010-03-03	19:35.00000000
2010-03-03 19:36:00	2010-03-03	19:36.00000000

Para ajustar el formato de la hora y omitir los segundos, conservando únicamente las horas y los minutos (HH:mm), se ejecuta el siguiente script:

```
-- 1. Cambiar el tipo de dato de la columna para que acepte texto con la estructura 'HH:mm'
ALTER TABLE transactions_data
ALTER COLUMN hora VARCHAR(5);
GO

-- 2. Actualizar los registros guardando únicamente los 4 dígitos (horas y minutos)
UPDATE transactions_data
SET hora = FORMAT(CAST(hora AS TIME), N'hh:mm');
GO
```

Se comprueba que efectivamente se ha ejecutado bien.

(2551145 rows affected)

Completion time: 2026-09-05T12:05:52.3784516+02:00

1.2 Creación de nueva tabla y modificación de merchant_state

Para estructurar y limpiar el núcleo transaccional, se creó la tabla definitiva `transactions_final` dotada de una clave primaria en `transaction_id`.

-- 1º Creación de la estructura limpia con clave primaria

```
CREATE TABLE transactions_final (  
    transaction_id INT PRIMARY KEY,  
    client_id INT,  
    card_id INT,  
    amount DECIMAL(10, 2),  
    fecha DATE,  
    hora TIME,  
    use_chip VARCHAR(50),  
    merchant_id INT,  
    merchant_city VARCHAR(100),  
    merchant_state VARCHAR(50),  
    zip VARCHAR(10),  
    mcc VARCHAR(10),  
    error_type VARCHAR(50),  
    pais VARCHAR(50)  
);  
GO
```

El siguiente paso, incluye la homologación de la columna `merchant_state`, reemplazando abreviaturas por el nombre completo de la región (e.g., 'CA' por 'California'); la depuración del campo `zip` para remover los decimales residuales de la importación; y la incorporación de la columna `pais` con el fin de clasificar las operaciones digitales bajo la categoría 'Online / Unknown'. Finalmente, se define `transaction_id` como clave primaria (PRIMARY KEY) para preservar la integridad referencial de la entidad."

-- 2º Inserción y transformación de columnas

```
INSERT INTO transactions_final (  
    transaction_id, client_id, card_id, amount, fecha, hora,  
    use_chip, merchant_id, merchant_city, merchant_state, zip, mcc, error_type, pais  
)  
SELECT  
    id AS transaction_id,  
    client_id,  
    card_id,  
    amount,  
    CAST([date] AS DATE) AS fecha,  
    CAST([date] AS TIME) AS hora,  
    use_chip,  
    merchant_id,  
    merchant_city,  
    CASE WHEN merchant_state = 'CA' THEN 'California' ELSE merchant_state END AS merchant_state,  
    LEFT(CAST(zip AS VARCHAR(20)), CHARINDEX('.', CAST(zip AS VARCHAR(20))) - 1) AS zip,  
    mcc,  
    errors AS error_type,  
    CASE WHEN merchant_city = 'ONLINE' OR merchant_state IS NULL THEN 'Online / Unknown' ELSE 'United States' END AS pais  
FROM transactions_data;
```

En el conjunto de datos original, la columna `merchant_state` contiene los estados registrados únicamente mediante sus abreviaturas postales de dos caracteres (por ejemplo, 'CA', 'AL', 'NY').

merchant_city	merchant_state	zip	mcc	errors
Rancho Cucamonga	CA	91730.0	5812	
Odenville	AI	35120.0	5310	
Ukiah	CA	95482.0	5411	
Granville	IA	51022.0	5499	
Monsey	NY	10952.0	5311	
Albany	NY	12203.0	5411	
ONLINE			4121	
Farmington	MN	55024.0	5499	
Detroit	MI	48235.0	5814	
ONLINE			4121	
Bethesda	MD	20814.0	3390	
Barbourville	KY	40906.0	4829	Bad PIN

Para estandarizar estos valores y reemplazar las abreviaturas por la denominación completa de cada estado, se implementa una estructura condicional **CASE**:

CASE merchant_state	merchant_city	merchant_state
WHEN 'AL' THEN 'Alabama'	Manchester	New Hampshire
WHEN 'AK' THEN 'Alaska'	Allenton	Michigan
WHEN 'AZ' THEN 'Arizona'	Bakersfield	California
WHEN 'AR' THEN 'Arkansas'	San Diego	California
WHEN 'CA' THEN 'California'	Belleville	Illinois
WHEN 'CO' THEN 'Colorado'	Mcalester	Oklahoma
WHEN 'CT' THEN 'Connecticut'	Bronx	New York
WHEN 'DE' THEN 'Delaware'	Seguin	Texas
WHEN 'FL' THEN 'Florida'	Dallas	Texas
WHEN 'GA' THEN 'Georgia'	Annandale	Virginia
WHEN 'HI' THEN 'Hawaii'	Claypool	Indiana
WHEN 'ID' THEN 'Idaho'	San Jose	California
WHEN 'IL' THEN 'Illinois'	Canton	North Carolina
WHEN 'IN' THEN 'Indiana'	Olympia	Washington
WHEN 'IA' THEN 'Iowa'	Kaneohe	Hawaii
WHEN 'KS' THEN 'Kansas'	Roseville	California
WHEN 'KY' THEN 'Kentucky'	Bremerton	Washington
WHEN 'LA' THEN 'Louisiana'	Watkinsville	Georgia
WHEN 'ME' THEN 'Maine'		
WHEN 'MD' THEN 'Maryland'		
WHEN 'MA' THEN 'Massachusetts'		
WHEN 'MI' THEN 'Michigan'		
WHEN 'MN' THEN 'Minnesota'		
WHEN 'MS' THEN 'Mississippi'		
WHEN 'MO' THEN 'Missouri'		
WHEN 'MT' THEN 'Montana'		

1.3 Crear columna país en base a la de merchant_city

Para dotar al modelo analítico de una segmentación geográfica precisa y estandarizada, se implementó una regla de negocio condicional mediante estructuras **CASE WHEN** durante el volcado y transformación de la tabla **transactions_final**. Esta lógica evalúa de forma automatizada el campo de la ciudad del comercio (**merchant_city**), asignando el país correspondiente según la taxonomía internacional del proyecto.

Asimismo, el script clasifica bajo la etiqueta '**Online / Unknown**' aquellas operaciones de naturaleza digital o que carecen de un estado registrado, asignando por defecto '**United States**' al resto de establecimientos físicos locales. Este enriquecimiento de datos resulta fundamental para la posterior ejecución de consultas de riesgo geográfico y la detección de anomalías transaccionales transfronterizas.

```

CASE
WHEN merchant_city = 'ONLINE' OR merchant_state IS NULL THEN 'Online / Unknown'
WHEN merchant_city IN ('Athens', 'Thessaloniki') THEN 'Greece'
WHEN merchant_city IN ('Vienna', 'Salzburg') THEN 'Austria'
WHEN merchant_city IN ('Muscat') THEN 'Oman'
WHEN merchant_city IN ('Luxembourg') THEN 'Luxembourg'
WHEN merchant_city IN ('Abuja', 'Lagos') THEN 'Nigeria'
WHEN merchant_city IN ('Rome', 'Milan') THEN 'Italy'
WHEN merchant_city IN ('Kingston') THEN 'Jamaica'
WHEN merchant_city IN ('Tashkent') THEN 'Uzbekistan'
WHEN merchant_city IN ('Belmopan') THEN 'Belize'
WHEN merchant_city IN ('Port-au-Prince') THEN 'Haiti'
WHEN merchant_city IN ('Mexico City', 'Cancun') THEN 'Mexico'
WHEN merchant_city IN ('Toronto', 'Vancouver', 'Montreal') THEN 'Canada'
WHEN merchant_city IN ('Nairobi') THEN 'Kenya'
WHEN merchant_city IN ('Jerusalem', 'Tel Aviv') THEN 'Israel'
WHEN merchant_city IN ('Berlin', 'Munich', 'Frankfurt') THEN 'Germany'
WHEN merchant_city IN ('Bangkok') THEN 'Thailand'
WHEN merchant_city IN ('Budapest') THEN 'Hungary'
WHEN merchant_city IN ('Chisinau') THEN 'Moldova'
WHEN merchant_city IN ('Istanbul', 'Ankara') THEN 'Turkey'
WHEN merchant_city IN ('Bucharest') THEN 'Romania'
ELSE 'United States'
END AS pais

```

1.4 Rellenar las celdas vacías de la columna error

Para asegurar la consistencia y limpieza de los registros en la columna `error_type`, se implementó la función nativa `ISNULL()` durante el volcado de datos. Esta instrucción evalúa los campos vacíos o nulos procedentes del sistema de origen, sustituyéndolos de forma automatizada por la cadena 'No Error'. Gracias a esta transformación, se evita la presencia de valores indefinidos en la pasarela de pagos, estandarizando el formato descriptivo de las incidencias técnicas para su posterior análisis estadístico.

```
ISNULL(errors, 'No Error') AS error_type,
```

La tabla final `transactions_final` organiza los datos en una secuencia analítica estratégica —iniciando con la identidad del cliente, métricas financieras, separación precisa de fecha y hora, jerarquía geográfica completa y diagnóstico de errores—, lo cual resulta crítico para la detección de fraude al optimizar el cálculo de velocidad transaccional, identificar anomalías de desplazamiento físico imposible, aislar comercios de alto riesgo mediante el código `mcc` y correlacionar intentos fallidos

previos con patrones de vulneración de tarjetas.

	transaction_id	client_id	card_id	amount	fecha	hora	use_chip	merchant_id	merchant_city	merchant_state	zip	mcc	error_type	pais
1	7805495	428	3842	33.62	25-03-2010	11.47	Swipe Transaction	49194	Manchester	New Hampshire	3104	5813	No Error	United States
2	7805496	692	4303	18.05	25-03-2010	11.47	Swipe Transaction	87625	Allenton	Michigan	48002	5812	No Error	United States
3	7805497	1002	1149	77.74	25-03-2010	11.47	Swipe Transaction	61195	Bakersfield	California	93306	5541	No Error	United States
4	7805498	1016	5883	4.67	25-03-2010	11.47	Swipe Transaction	34249	San Diego	California	92130	5310	No Error	United States

1.5 ETL. Transformación de los datos

Validación de la Tabla de User_data

```

SELECT
    SUM(CASE WHEN id IS NULL THEN 1 ELSE 0 END) AS nulos_id,
    SUM(CASE WHEN current_age IS NULL THEN 1 ELSE 0 END) AS nulos_current_age,
    SUM(CASE WHEN retirement_age IS NULL THEN 1 ELSE 0 END) AS nulos_retirement_age,
    SUM(CASE WHEN birth_year IS NULL THEN 1 ELSE 0 END) AS nulos_birth_year,
    SUM(CASE WHEN birth_month IS NULL THEN 1 ELSE 0 END) AS nulos_birth_month,
    SUM(CASE WHEN gender IS NULL THEN 1 ELSE 0 END) AS nulos_gender,
    SUM(CASE WHEN address IS NULL THEN 1 ELSE 0 END) AS nulos_address,
    SUM(CASE WHEN per_capita_income IS NULL THEN 1 ELSE 0 END) AS nulos_per_capita_income,
    SUM(CASE WHEN yearly_income IS NULL THEN 1 ELSE 0 END) AS nulos_yearly_income,
    SUM(CASE WHEN total_debt IS NULL THEN 1 ELSE 0 END) AS nulos_total_debt,
    SUM(CASE WHEN credit_score IS NULL THEN 1 ELSE 0 END) AS nulos_credit_score,
    SUM(CASE WHEN num_credit_cards IS NULL THEN 1 ELSE 0 END) AS nulos_num_credit_cards
FROM users_data;

```

	nulos_id	nulos_current_age	nulos_retirement_age	nulos_birth_year	nulos_birth_month	nulos_gender	nulos_address	nulos_per_capita_income	nulos_yearly_income	nulos_total_debt	nulos_credit_score	nulos_num_credit_cards
1	0	0	0	0	0	0	0	0	0	0	0	0

Para auditar la integridad y calidad estructural de la tabla de usuarios (**users_data**), se ejecutaron consultas de control destinadas a la detección exhaustiva de valores nulos y registros duplicados. Tras el análisis sistemático mediante funciones de agregación y conteo condicional, se comprobó que la entidad se encuentra completamente íntegra, sin la presencia de campos vacíos ni claves primarias duplicadas, lo que valida la calidad de los datos sociodemográficos de origen y permite avanzar con seguridad hacia el procesamiento de la siguiente entidad relacional.

```

SELECT id, COUNT(*) AS total_duplicados
FROM users_data
GROUP BY id
HAVING COUNT(*) > 1;

```

id	total_duplicados
----	------------------

Validación de la Tabla Card_data

Al encontrarse la tabla de usuarios completamente limpia de nulos y duplicados, el siguiente paso lógico

es auditar y estructurar la tabla de instrumentos financieros (**cards_data**).

Para revisar los valores nulos en esta tabla, aplicamos la siguiente consulta de auditoría:

```
SELECT
    SUM(CASE WHEN id IS NULL THEN 1 ELSE 0 END) AS nulos_id,
    SUM(CASE WHEN client_id IS NULL THEN 1 ELSE 0 END) AS nulos_client_id,
    SUM(CASE WHEN card_brand IS NULL THEN 1 ELSE 0 END) AS nulos_card_brand,
    SUM(CASE WHEN card_type IS NULL THEN 1 ELSE 0 END) AS nulos_card_type,
    SUM(CASE WHEN credit_limit IS NULL THEN 1 ELSE 0 END) AS nulos_credit_limit
FROM cards_data;
```

	nulos_id	nulos_client_id	nulos_card_brand	nulos_card_type	nulos_credit_limit
1	0	0	0	0	0

Siguiendo la misma metodología de auditoría de calidad de datos, se examinó la tabla de instrumentos financieros (**cards_data**) mediante consultas de control para la detección de valores nulos y registros duplicados. Tras la ejecución del análisis, se constató la inexistencia de anomalías estructurales, arrojando un balance de cero valores nulos y cero duplicados, lo que valida la alta calidad, consistencia e integridad de la información de tarjetas previa a su integración en el modelo definitivo.

```
SELECT id, COUNT(*) AS total_duplicados
FROM cards_data
GROUP BY id
HAVING COUNT(*) > 1;
```

id	total_duplicados
----	------------------

Validar Tabla Mcc

Al encontrarse la tabla de usuarios completamente limpia de nulos y duplicados, el siguiente paso lógico es auditar y estructurar la tabla de instrumentos financieros (**Mcc_codes**).

```
SELECT
    SUM(CASE WHEN cmm IS NULL THEN 1 ELSE 0 END) AS nulos_mcc,
    SUM(CASE WHEN descripcion IS NULL THEN 1 ELSE 0 END) AS nulos_descripcion
FROM mcc_codes
```

	nulos_mcc	nulos_description
1	0	0

Continuando con la validación de las tablas, se auditó el catálogo de categorías comerciales (**mcc_codes**) para verificar su consistencia interna. Tras la ejecución de las consultas de control orientadas a la detección de valores nulos y códigos duplicados, se obtuvo un resultado de cero incidencias, lo que confirma que la información de los códigos MCC se encuentra completa, normalizada y libre de redundancias para su correcta vinculación relacional en el modelo de análisis.

```
SELECT cmm, COUNT(*) AS total_duplicados
FROM mcc_codes
GROUP BY cmm
HAVING COUNT(*) > 1;
```

cmm	total_duplicados
-----	------------------

Resultados

Tras las auditorías exhaustivas realizadas sobre las tablas compuestas por la database (<https://www.kaggle.com/datasets/computingvictor/transactions-fraud-datasets>) se comprobó que los datos originales extraídos de la plataforma presentaban una limpieza absoluta, sin rastro de valores nulos ni duplicados. Por este motivo, dichas entidades no requirieron modificaciones estructurales ni scripts de depuración adicionales.

Toda la fase de transformación y normalización del proyecto se focalizó de manera exclusiva en la tabla transaccional principal (**transactions_final**), siendo esta la única sección que demandó intervenciones de limpieza como la división de fecha y hora, la estandarización de códigos postales, el tratamiento de errores mediante valores predeterminados y el enriquecimiento geográfico a través de reglas condicionales.

Paso 4: Establecer las relaciones del modelo (Llaves Foráneas - Foreign Keys)

Esta acción es indispensable para garantizar la integridad referencial y permitir vincular correctamente las entidades al momento de realizar las consultas analíticas del proyecto. Se configuran las llaves foráneas entre usuarios, tarjetas, transacciones y etiquetas de fraude.

Para ello, se utilizarán las siguientes instrucciones:

```
ALTER TABLE cards_data
ADD CONSTRAINT FK_Cards_Users FOREIGN KEY (client_id) REFERENCES users_data(id);

ALTER TABLE transactions_final
```

```

ADD CONSTRAINT FK_Transactions_Cards FOREIGN KEY (card_id) REFERENCES cards_data(id);

ALTER TABLE transactions_final
ADD CONSTRAINT FK_Transactions_Users FOREIGN KEY (client_id) REFERENCES users_data(id);

ALTER TABLE transactions_final
ADD CONSTRAINT FK_Transactions_MCC FOREIGN KEY (mcc) REFERENCES mcc_codes(cmm);

ALTER TABLE Train_Fraud_Labels
ADD CONSTRAINT FK_Fraud_Transactions FOREIGN KEY (Transaction_ID) REFERENCES
transactions_final(transaction_id);
GO

```

Paso 5: Consulta 1 – Detección de transacciones atípicas mediante Funciones de Ventana

Se implementa una Expresión de Tabla Común (CTE) con funciones de ventana (SUM OVER, AVG OVER, PERCENT_RANK OVER) para calcular el gasto acumulado por tarjeta, el promedio histórico del cliente y su percentil de gasto. Se aíslan las operaciones ubicadas en el percentil 95 o superior para contrastarlas contra la columna **is_fraud**.

Para ello, se utilizarán las siguientes instrucciones:

```

WITH AnalisisVentana AS (
    SELECT
        f.transaction_id,
        f.client_id,
        f.card_id,
        f.amount,
        f.fecha,
        f.hora,
        t.is_fraud,
        SUM(f.amount) OVER (
            PARTITION BY f.card_id ORDER BY f.fecha, f.hora
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) AS Gasto_Acumulado_Tarjeta,
        AVG(f.amount) OVER (PARTITION BY f.client_id) AS Promedio_Gasto_Cliente,
        PERCENT_RANK() OVER (PARTITION BY f.client_id ORDER BY f.amount ASC) AS
Ranking_Percentil_Monto
    FROM transactions_final AS f
    INNER JOIN Train_Fraud_Labels AS t ON f.transaction_id = t.Transaction_ID
    WHERE f.amount > 0
)
SELECT TOP 50
    transaction_id, client_id, card_id, amount, fecha, hora,
    Gasto_Acumulado_Tarjeta,
    ROUND(Promedio_Gasto_Cliente, 2) AS Promedio_Gasto_Cliente,
    ROUND(Ranking_Percentil_Monto * 100, 2) AS Percentil_Monto,

```

```

    is_fraud
FROM AnalisisVentana
WHERE Ranking_Percentil_Monto >= 0.95
ORDER BY amount DESC;

```

	transaction_id	client_id	card_id	amount	fecha	hora	Gasto_Acumulado_Tarjeta	Promedio_Gasto_Cliente	Percentil_Monto	is_fraud
1	7808164	1142	4674	4978.45	26-03-2010	07.00	39321.49	88.860000	100	Yes
2	9662810	96	3773	4685.23	03-06-2011	14.24	65303.19	97.170000	100	No
3	8877661	1983	3399	16548.50	01-12-2010	04.34	16548.50	67.580000	100	No
4	8431907	719	5518	4199.33	25-08-2010	21.02	43785.65	50.330000	100	No
5	7686947	776	5859	4004.73	23-02-2010	15.22	81045.21	111.000000	100	No
6	7871493	1307	111	3854.44	11-04-2010	03.21	16703.31	92.870000	100	Yes
7	7792285	1746	3754	3750.60	22-03-2010	08.27	6180.39	97.520000	100	Yes
8	8197899	708	5165	3424.51	30-06-2010	10.48	70564.57	150.990000	100	No
9	8766245	1495	2922	3416.64	14-11-2010	11.51	14462.53	61.330000	100	Yes

El análisis evidencia que, si bien el filtro por percentiles aísla eficazmente los picos históricos y detecta una marcada desviación histórica —con transacciones superiores a 4.000 euros frente a consumos habituales bajo los 100 euros—, el importe por sí solo es insuficiente para clasificar el fraude, dado el registro de falsos positivos en operaciones legítimas de cuantía similar; por tanto, se confirma que una cifra atípica no determina una actividad ilícita sin la incorporación de variables contextuales que refuercen la precisión del modelo.

Paso 5: Consulta 2 – Análisis matricial cruzado mediante el operador PIVOT

Se utiliza el operador PIVOT para transformar filas en columnas, cruzando la categoría comercial del establecimiento contra el tipo de tarjeta (Crédito vs. Débito) filtrando solo aquellas operaciones marcadas con **is_fraud = 'Yes'**. Esto permite identificar qué sectores y plásticos sufren mayor incidencia delictiva.

Para ello, se utilizarán las siguientes instrucciones:

```

SELECT
    Categoria_Comercio,
    ISNULL([Credit], 0) AS Tarjeta_Credito,
    ISNULL([Debit], 0) AS Tarjeta_Debito
FROM (
    SELECT
        c.descripcion AS Categoria_Comercio,
        d.card_type,
        f.transaction_id
    FROM transactions_final AS f
    INNER JOIN cards_data AS d ON f.card_id = d.id
    INNER JOIN mcc_codes AS c ON f.mcc = c.cmm
    INNER JOIN Train_Fraud_Labels AS t ON f.transaction_id = t.Transaction_ID
    WHERE t.is_fraud = 'Yes'
) AS BaseDatos
PIVOT (

```

```

COUNT(transaction_id)
FOR card_type IN ([Credit], [Debit])
) AS TablaPivote
ORDER BY (ISNULL([Credit], 0) + ISNULL([Debit], 0)) DESC;

```

	Categoria_Comercio	Tarjeta_Credito	Tarjeta_Debito
1	Department Stores	180	264
2	Wholesale Clubs	81	118
3	Discount Stores		
4	Electronics Stores	38	80
5	Drug Stores and Pharmacies	40	73
6	Digital Goods - Media, Books, Apps	37	70
7	Miscellaneous Home Furnishing Stores	38	43
8	Motor Freight Carriers and Trucking	26	33

El análisis revela una clara concentración sectorial de los incidentes en establecimientos de gran superficie como los *Department Stores*, clubes mayoristas y tiendas de descuento —superando los 400 casos totales al aprovechar el tráfico masivo de compradores para camuflar las transacciones ilícitas—, sumada a una marcada vulnerabilidad en las tarjetas de débito, las cuales presentan de forma sistemática un mayor volumen de operaciones fraudulentas que las de crédito debido a la asimetría en los protocolos de protección, garantías y coberturas de seguro provistos por las entidades financieras.

Paso 5: Consulta 3 – Agrupación demográfica y geográfica con GROUP BY

Cruzando la tabla **users_data** con las transacciones mediante INNER JOIN y GROUP BY, se analiza el impacto del fraude según el género del titular y el estado del comercio. Se calculan el total de siniestros, el importe global defraudado y el monto medio por operación.

Para ello, se utilizarán las siguientes instrucciones:

```

SELECT
    u.gender AS Genero_Cliente,
    f.merchant_state AS Estado_Comercio,
    COUNT(f.transaction_id) AS Total_Fraudes,
    SUM(f.amount) AS Monto_Total_Fraudulento,
    ROUND(AVG(f.amount), 2) AS Monto_Promedio_Fraude
FROM transactions_final AS f
INNER JOIN users_data AS u ON f.client_id = u.id
INNER JOIN Train_Fraud_Labels AS t ON f.transaction_id = t.Transaction_ID
WHERE t.is_fraud = 'Yes'
    AND f.merchant_state IS NOT NULL
GROUP BY u.gender, f.merchant_state
ORDER BY Total_Fraudes DESC;

```

	Genero_Cliente	Estado_Comercio	Total_Fraudes	Monto_Total_Fraudulento	Monto_Promedio_Fraude
1	Female		1217	148193.01	121.770000
2	Male		1140	142912.74	125.360000
3	Male	Haiti			
4	Female	Haiti	120	10965.18	91.380000
5	Female	Tuvalu	4	231.41	57.850000
6	Male	California	2	108.29	54.150000
7	Male	Tuvalu	1	8.30	8.300000

Para identificar patrones de riesgo asociados a la ubicación física del comercio y al perfil sociodemográfico del titular, se implementó una consulta de agregación mediante la cláusula **GROUP BY**, la cual consolida las transacciones fraudulentas confirmadas al cruzar el género del usuario (**gender**) con el estado del comercio (**merchant_state**); de esta forma, se calculan métricas esenciales como el volumen total de incidencias (**Total_Fraudes**), el impacto económico global (**Monto_Total_Fraudulento**) y el importe promedio por operación ilícita, permitiendo aislar focos geográficos críticos y determinar discrepancias conductuales significativas entre perfiles demográficos. La mayoría de los casos son Online.

Paso 5: Consulta 4 – Reconstrucción temporal con CROSS APPLY (Últimas 3 transacciones)

Para examinar el comportamiento previo a un evento fraudulento, se utiliza el operador **CROSS APPLY** junto con una subconsulta correlacionada acotada (TOP 3). Esto despliega las tres operaciones anteriores realizadas con la misma tarjeta, permitiendo identificar movimientos de prueba o inconsistencias geográficas inmediatas.

Para ello, se utilizarán las siguientes instrucciones:

```

SELECT
    t.Transaction_ID,
    f.client_id,
    f.card_id,
    f.amount AS Monto_Fraude,
    f.fecha AS Fecha_Fraude,
    UltimasOperaciones.transaction_id AS ID_Transaccion_Previa,
    UltimasOperaciones.amount AS Monto_Previo,
    UltimasOperaciones.fecha AS Fecha_Previa
FROM Train_Fraud_Labels AS t
INNER JOIN transactions_final AS f ON t.Transaction_ID = f.transaction_id
CROSS APPLY (
    SELECT TOP 3
        sub.transaction_id, sub.amount, sub.fecha, sub.hora
    FROM transactions_final AS sub
    WHERE sub.card_id = f.card_id
    AND (sub.fecha < f.fecha OR (sub.fecha = f.fecha AND sub.hora < f.hora))

```

```

ORDER BY sub.fecha DESC, sub.hora DESC
) AS UltimasOperaciones
WHERE t.is_fraud = 'Yes'
ORDER BY f.transaction_id, UltimasOperaciones.fecha DESC;

```

	Transaction_ID	client_id	card_id	Monto_Fraude	Fecha_Fraude	ID_Transaccion_Previa	Monto_Previo	Fecha_Previa
1	10547260	597	4117	93.85	03-01-2012	8841676	-88.00	02-12-2010
2	10547260	597	4117	93.85	03-01-2012	8841654	88.00	02-12-2010
3	10547260	597	4117	93.85	03-01-2012	8841625	53.40	02-12-2010
4	10547355	597	4117	28.55	03-01-2012	10547260	93.85	03-01-2012
5	10547355	597	4117	28.55	03-01-2012	8841676	-88.00	02-12-2010
6	10547355	597	4117	28.55	03-01-2012	8841654	88.00	02-12-2010
7	10551756	479	2186	77.24	04-01-2012	7490217	63.63	04-01-2010
8	10551756	479	2186	77.24	04-01-2012	10282621	103.43	03-11-2011
9	10551756	479	2186	77.24	04-01-2012	10281613	5.26	03-11-2011

Para analizar el comportamiento histórico inmediato previo a una operación catalogada como fraude (`is_fraud = 'Yes'`), se implementó una consulta avanzada basada en el operador **CROSS APPLY**. Mediante una subconsulta correlacionada limitada a los tres registros más cercanos (**TOP 3**) para un mismo instrumento de pago (`card_id`), el sistema extrae de forma estructurada la secuencia de transacciones previas. Este enfoque multiregistro otorga a los analistas de seguridad una visibilidad detallada de la racha operativa anterior, facilitando la detección temprana de patrones anómalos, comportamientos erráticos o indicios de suplantación previos a la consumación del evento delictivo.

Paso 5: Consulta 5 – Clientes de alto riesgo con Subconsultas Complejas (EXISTS y HAVING)

Esta consulta combina dos niveles de subconsultas avanzadas para aislar clientes críticos: una subconsulta correlacionada en el WHERE con EXISTS (que valida transacciones en comercios online) y una subconsulta escalar en el HAVING para filtrar aquellos titulares cuyo importe promedio de fraude supera la media global de toda la base de datos.

Para ello, se utilizarán las siguientes instrucciones:

```

SELECT
    u.id AS Cliente_ID,
    u.gender AS Genero,
    u.current_age AS Edad,
    u.credit_score AS Score_Crediticio,
    COUNT(f.transaction_id) AS Cantidad_Fraudes,
    SUM(f.amount) AS Monto_Total_Perdido,
    ROUND(AVG(f.amount), 2) AS Promedio_Por_Fraude
FROM users_data AS u
INNER JOIN transactions_final AS f ON u.id = f.client_id
INNER JOIN Train_Fraud_Labels AS t ON f.transaction_id = t.Transaction_ID
WHERE t.is_fraud = 'Yes'

```

```

AND EXISTS (
    SELECT 1 FROM transactions_final AS tf_sub
    WHERE tf_sub.client_id = u.id AND tf_sub.pais = 'Online / Unknown'
)
GROUP BY u.id, u.gender, u.current_age, u.credit_score
HAVING AVG(f.amount) > (
    SELECT AVG(tf_global.amount)
    FROM transactions_final AS tf_global
    INNER JOIN Train_Fraud_Labels AS tl_global ON tf_global.transaction_id =
tl_global.Transaction_ID
    WHERE tl_global.is_fraud = 'Yes'
)
ORDER BY Monto_Total_Perdido DESC;

```

	Ciente_ID	Genero	Edad	Score_Crediticio	Cantidad_Fraudes	Monto_Total_Perdido	Promedio_Por_Fraude
1	1142	Male	52	716	17	6763.85	397.870000
2	1746	Female	53	701	13	5125.89	394.300000
3	461	Female	42	784	10	4693.92	469.390000
4	46	Male	62	632	19	4538.66	238.880000
5	1307	Female	52	646	8	4474.40	559.300000
6	1495	Male	52	770	8	4207.29	525.910000
7	1569	Female	68	722	18	4060.79	225.600000
8	323	Male	54	683	13	3968.08	305.240000
9	1805	Male	38	718	7	3708.77	529.820000
10	604	Male	60	736	7	3289.28	469.900000
11	1190	Female	49	707	26	3231.12	124.270000
12	502	Male	46	745	8	3216.55	402.070000
13	845	Male	58	786	15	3174.87	211.660000
14	1100	Male	61	573	14	3151.98	225.140000

Esta consulta implementa un análisis agregado y multifactorial para identificar patrones de gasto elevado y su relación con el fraude, combinando información transaccional, demográfica, de tarjetas, de comercios y de etiquetas de fraude.

Integra la tabla principal de transacciones con cuatro dimensiones clave: la descripción de la categoría el comercio, los datos de la tarjeta, el perfil sociodemográfico del usuario y el indicador de fraude